Spec and test gaps: AON timer

Repository `targets/opentitan` at commit `b84154c71810`. Report generated 2026-11-10 22:09 UTC by chipbag.

Each finding below gives the gap, the evidence for it as exact quotes at their file and line, and how it was verified: an agent searched the repository to refute the finding, and the finding was kept only if that failed. Every quote was checked against the file at the cited lines. The report does not recommend a fix.

Summary

#	Category	Verified	Withdrawn	Inconclusive
1	Spec too ambiguous to test	3	5	0
2	Spec contradicts itself	1	1	0
3	Test contradicts its own description	5	0	0
4	Spec and test disagree	0	0	0
5	Spec not tested	12 (26 items)	8	4

1 verified finding with the same subject as another are shown together with it and counted once.

Scope

Spec sources (4): `hw/ip/aon_timer/data/aon_timer.hjson`, `hw/ip/aon_timer/doc/interfaces.md`, `hw/ip/aon_timer/doc/programmers_guide.md`, `hw/ip/aon_timer/doc/theory_of_operation.md`

Verification sources (41 files): `hw/ip/aon_timer/dv/env/seq_lib/` (11), `hw/dv/tools/dvsim/testplans/` (6), `sw/device/tests/` (5), `hw/ip/aon_timer/dv/cov/` (4), `hw/ip/aon_timer/dv/env/` (4), `hw/ip/aon_timer/data/` (2), `hw/ip/aon_timer/dv/` (2), `hw/top_earlgrey/dv/` (2), `hw/ip/aon_timer/dv/sva/` (1), `hw/ip/aon_timer/dv/tests/` (1), `hw/top_earlgrey/data/ip/` (1), `hw/top_earlgrey/formal/conn_csvs/` (1), `sw/device/lib/dif/` (1). The full list is in the manifest.

143 requirements after merging duplicates; 87 need a test (the rest hold by construction). 86 tests (70 with source), 50 test plan items. 47 requirements are matched to a test.

Files the verifier read that are not in the manifest: `hw/dv/sv/tl_agent/tl_agent_cov.sv`, `hw/dv/tools/dvsim/tests/csr_tests.hjson`, `hw/ip/aon_timer/README.md`, `hw/ip/aon_timer/doc/programmers_guide.md`, `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv`, `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_alternating_enable_on_off_vseq.sv`, `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_prescaler_vseq.sv`, `hw/ip/aon_timer/dv/sva/aon_timer_bind.sv`, `sw/device/tests/aon_timer_wdog_lc_escalate_test.c`.

1. Spec too ambiguous to test

1.1 A watchdog bite reset is generated when the watchdog count reaches the threshold value.

`f_f2608a62b470` · weak modal · severity medium · section: hw/ip/aon_timer/data/aon_timer.hjson > aon_timer > WDOG_BITE_THOLD

Gap: The text uses 'should be generated' and does not say whether the reset triggers on equality or on greater-or-equal.

Spec says:

- `hw/ip/aon_timer/data/aon_timer.hjson:303`: `The count at which a watchdog bite reset should be generated.`

Not answered by the spec: Is the watchdog bite condition count >= WDOG_BITE_THOLD.threshold or count == threshold? If the count is already above a newly programmed threshold, or software writes the count above it, does a bite assert immediately? Also, in which clk_aon cycle does the bite request assert relative to the count reaching the threshold?

Verified: No spec text states whether the watchdog bite condition is >= or ==, or the cycle at which the bite asserts. The only supporting text is the programmer's guide, which covers the wakeup timer only. The RTL uses >= gated by an increment pulse.

Searched:

- bite, threshold, >= wording in the aon_timer docs in `hw/ip/aon_timer/doc`: Only generic 'reaches the programmed threshold' text. The wakeup 'remains above' sentence is the only >= hint, and it does not cover the watchdog.
- comparison operators in the RTL in `hw/ip/aon_timer/rtl`: The RTL uses >= for the bite request, gated by wdog_incr.

Looked at and ruled out:

- `hw/ip/aon_timer/doc/programmers_guide.md:93`: It covers only the wakeup timer, not the watchdog bite, and does not give the cycle timing.

1.2 The interrupt retriggers at the next clock tick if the count remains above the threshold after clearing and the timer remains enabled.

`f_604ea5c51516` · unclear referent · severity medium · section: hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Interrupt Handling

Gap: It is unclear which clock's tick is meant for retriggering.

Spec says:

- `hw/ip/aon_timer/doc/programmers_guide.md:93`

  > If {[`WKUP_COUNT_HI`](registers.md#wkup_count_hi), [`WKUP_COUNT_LO`](registers.md#wkup_count_lo)} remains above the threshold after clearing the interrupt or wakeup event and the timer remains enabled, the interrupt and wakeup event will trigger again at the next clock tick.

Not answered by the spec: After INTR_STATE/WKUP_CAUSE is cleared while the timer is enabled and {WKUP_COUNT_HI, WKUP_COUNT_LO} is still at or above WKUP_THOLD, is the next clock tick the next AON clock cycle or the next prescaled counter increment? How many AON and bus clock cycles of latency, including CDC synchronization, should a test allow before the interrupt and wakeup request re-assert? Is the retrigger condition count >= threshold or strictly count > threshold?

Verified: No other spec text in aon_timer docs specifies which clock tick causes retriggering, the CDC latency, or whether the comparison is >= or >. The theory of operation only mentions the ~200KHz AON clock and async register sync.

Searched:

- tick/prescal/threshold in aon_timer docs in `hw/ip/aon_timer/doc`: No definition of retrigger timing or comparison semantics
- clock/cycle/>=/reaches in theory_of_operation.md in `hw/ip/aon_timer/doc/theory_of_operation.md`: Only general clock and CDC remarks

Looked at and ruled out:

- `hw/ip/aon_timer/doc/registers.md:74`: 'The count at which a wakeup interrupt should be generated' does not define retrigger timing or the clock tick.

1.3 A register write can complete on the bus before the data has reached the underlying register in the AON domain.

`f_bfbfd9b86252` · missing detail · severity medium · section: hw/ip/aon_timer/doc/theory_of_operation.md > Theory of Operation > Design Details

Gap: No bound on the delay before the write reaches the AON register is given.

Spec says:

- `hw/ip/aon_timer/doc/theory_of_operation.md:26`: `This means that writes can complete before the data has reached its underlying register in the slow clock domain.`

Not answered by the spec: What is the maximum number of AON (and SYS) clock cycles between a TL-UL write completing and the value taking effect in the AON-domain register? Is any bound guaranteed beyond read-back ordering?

Verified: No spec text in the aon_timer docs or register descriptions bounds the delay. The only guarantee is that a read-back completes all earlier writes. The spec gives no cycle count.

Searched:

- slow clock, cycles, synchron in `hw/ip/aon_timer md/hjson`: Only the cited lines and generic DV README mentions. No bound.
- read-back, take effect, latency in `hw/ip/aon_timer md/hjson`: Only the cited read-back sentence.

Looked at and ruled out:

- `hw/ip/aon_timer/dv/README.md:36`: Only says there is synchronisation logic between the clock domains. It is a verification doc and gives no bound.

---

2. Spec contradicts itself

2.1 Wakeup threshold bits [31:0] are in the low threshold register.

`f_9cdb47e2d305` · description vs. definition · severity high

Gap: The text names WKUP_THOLD_HI for bits [31:0] while the link target is wkup_thold_lo, so the register is unclear.

Spec says:

- `hw/ip/aon_timer/doc/theory_of_operation.md:32`: `Bits[31:0] in registers`
- `hw/ip/aon_timer/doc/theory_of_operation.md:32`: `WKUP_THOLD_HI`

Not answered by the spec: Is the register holding wakeup threshold bits [31:0] WKUP_THOLD_LO?

Verified: Line 32 of the doc lists WKUP_THOLD_HI as the register for bits [31:0], but its link target is registers.md#wkup_thold_lo. Line 31 already assigns WKUP_THOLD_HI to bits [63:32]. The two readings are incompatible, and the link target and the 64-bit split point to LO being intended. It is a doc typo. I did not check the register definitions or RTL.

Further evidence found by the verifier:

- `hw/ip/aon_timer/doc/theory_of_operation.md:31`: `Bits[63:32] in registers [\`WKUP_COUNT_HI\`](registers.md#wkup_count_hi) and [\`WKUP_THOLD_HI\`](registers.md#wkup_thold_hi).` — HI is already used for bits [63:32].

Searched:

- context of the cited line in `hw/ip/aon_timer/doc/theory_of_operation.md:25-40`: Line 32 names WKUP_THOLD_HI with a link to wkup_thold_lo; line 31 uses HI for bits [63:32].

Same subject, also found as spec contradicts itself (`f_d086aa20d344`, cross_document):

Spec says:

- `hw/ip/aon_timer/data/aon_timer.hjson:206`: `bits: "31:0"`

Verified: The theory_of_operation.md line 32 says bits[31:0] are in WKUP_THOLD_HI, although its link target is wkup_thold_lo. Line 31 of the same file and the hjson say HI holds bits 63:32. This is a documentation typo, not a real ambiguity, but the text does contradict itself.

---

3. Test contradicts its own description

3.1 aon_timer_prescaler: plan vs. test

`f_39e92ca16bb1` · plan vs. test · severity medium

Gap: Plan item p2 says the test switches the WKUP prescaler at random times and checks that WKUP_COUNT still reflects the passing of real time (p2.k1). The test writes the prescaler once, before enabling the timer, and never changes it while the timer runs. It never reads or compares WKUP_COUNT. The only check is a backdoor spinwait that the prescaler field was written, and the test otherwise just waits

for the interrupt. Both the "switch at random times" stimulus and the WKUP_COUNT real-time check are missing.

Labels in this finding: p2 = `hw/ip/aon_timer/data/aon_timer_testplan.hjson`; p2.k1 = `hw/ip/aon_timer/data/aon_timer_testplan.hjson:44-45`

Test description says:

- `hw/ip/aon_timer/data/aon_timer_testplan.hjson:44-45`: `WKUP_COUNT should reflect expected -and changed- passing of real time regardless of its changing prescaler value.` — WKUP_COUNT reflects the expected, changed, passing of real time regardless of the changing prescaler value.

Test does:

- `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_prescaler_vseq.sv:57`: `csr_utils_pkg::csr_wr(ral.wkup_ctrl.prescaler, prescaler);` — stimulus; Write the random prescaler value to WKUP_CTRL.
- `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_prescaler_vseq.sv:46`: `wait_for_interrupt();` — stimulus; Wait for the interrupt.
- `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_prescaler_vseq.sv:58`: `csr_utils_pkg::csr_spinwait(.ptr(ral.wkup_ctrl.prescaler), .exp_data(prescaler), .backdoor(1));` — check; Spin-wait until the backdoor value of the prescaler field equals the written prescaler, confirming it propagated through the CDC. Failure handling (e.g., timeout) is not visible here.

Verified: The plan ties the prescaler item to aon_timer_prescaler. It promises prescaler switches at random times while the timer runs, with WKUP_COUNT checked. The vseq writes the prescaler once before enabling the timer and never changes it afterwards. It never reads WKUP_COUNT. The scoreboard does model wakeup timing with the prescaler, so the interrupt timing is checked indirectly. That does not supply the "switch while running" stimulus.

Searched:

- prescaler in the testplan in `aon_timer_testplan.hjson`: The prescaler item links to the test aon_timer_prescaler.
- prescaler in the scoreboard in `aon_timer_scoreboard.sv`: The scoreboard uses the prescaler in its wakeup timing model, so the interrupt timing is checked.

Looked at and ruled out:

- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:934`: The scoreboard checks wakeup timing generically. It does not give the test the missing stimulus of changing the prescaler at random times while the timer runs.

3.2 aon_timer_wdog_lc_escalate_test: check cannot fail

`f_eeb05247ba86` · check cannot fail · severity medium

Gap: The plan says the test fails if the bark interrupt fires in the interrupt handler, and the comment says no AON timer interrupts should occur. The handler only logs an error (LOG_ERROR) and does not fail the test, so an unexpected bark interrupt cannot fail the test.

Test description says:

- `hw/top_earlgrey/data/ip/chip_aon_timer_testplan.hjson:122-123`: `fail the test in the interrupt handler in case the bark interrupt is fired.` — The test fails if the bark interrupt fires in the interrupt handler.
- `sw/device/tests/aon_timer_wdog_lc_escalate_test.c:86`: `We should not get aon timer interrupts since escalation suppresses them.`

Test does:

- `sw/device/tests/aon_timer_wdog_lc_escalate_test.c:89`: `LOG_ERROR("Unexpected aon timer interrupt %d", irq);` — check; does not fail the test; An AON timer interrupt is unexpected. It is only logged as an error and not otherwise checked.

Verified: The AON timer branch of ottf_handle_irq only calls LOG_ERROR and returns true. It has no CHECK or other failure path. The test can still pass through the escalation reset path (second boot returns true) even if a bark interrupt fired. This contradicts the plan's "fail the test in the interrupt handler" and the comment's "should not get aon timer interrupts". I did not search for an external harness that fails on ERROR log lines.

Searched:

- Read the full test source for any CHECK or flag set on AON timer interrupts in `sw/device/tests/aon_timer_wdog_lc_escalate_test.c`: None. The only AON timer handling is LOG_ERROR, and there is no counter or flag that is checked later.

Looked at and ruled out:

- `sw/device/tests/aon_timer_wdog_lc_escalate_test.c:103-117`: These CHECKs are in the alert handler branch, not the AON timer branch.

3.3 run_wdog_bark_timer: comment vs. code

`f_6b8c0b2b572c` · comment vs. code · severity low

Gap: The comment says the code checks the reset_req pins. The code under it compares the predicted wkup_cause with the wakeup request pin (aon_intr_vif pin idx 1), not a reset request. The comment is misleading about which output is verified.

Test description says:

- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:1276`: `// Check reset_req pins:`

Test does:

- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:1282`: `` `DV_CHECK_CASE_EQ(predicted_wkup_req, cfg.aon_intr_vif.sample_pin(.idx(1))) `` — check; When enabled and wkup_cause was not written in the same cycle, compares predicted wkup_cause with the wakeup request pin.
- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:1284`: `else if (!(cfg.aon_intr_vif.sample_pin(.idx(1)) inside {0, predicted_wkup_req})) begin` — check;

When wkup_cause was written in the same cycle, the wakeup request pin must be either 0 or the predicted value, otherwise fatal.

- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:1287`: `` `uvm_fatal(`gfn, $sformatf("aon_wkup_req_o comparison failed (not matching 0/%0d)", `` — check; When wkup_cause was written in the same cycle, the wakeup request pin must be either 0 or the predicted value, otherwise fatal.

Verified: The comment says "Check reset_req pins" but the code checks the wakeup request output (aon_intr_vif pin idx 1, aon_wkup_req_o) against wkup_cause. The check is real and can fail, so only the comment is wrong. The code is in model_and_check_wdog_bark, called from run_wdog_bark_timer. Low severity.

Searched:

- Read the code around the comment in `aon_timer_scoreboard.sv:1240-1300`: Only wkup_req (pin idx 1) is compared; no reset_req check is present in that block.

3.4 predict_wkup_cause: comment vs. code

`f_8f5bd34a107e` · comment vs. code · severity low

Gap: The comment says the code blocks only if 'predicting_value' = 1, but the code that follows is gated on `wait_for_we`. No 'predicting_value' appears in the shown code, so the comment looks stale or refers to a different condition.

Test description says:

- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:368`: `// Blocks only if 'predicting_value' = 1`

Test does:

- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:369-370` — stimulus; Optionally wait for the wkup_cause write-enable pulse.

```
if (wait_for_we)
                cfg.wait_for_we_pulse(".u_reg.aon_wkup_cause_we");
```

Verified: The comment mentions 'predicting_value', but the code is gated on the `wait_for_we` argument. 'predicting_value' appears nowhere else in the aon_timer DV code, so the comment is stale. This is cosmetic and has no functional impact.

Searched:

- predicting_value in `hw/ip/aon_timer/dv`: Only the comment at line 368; no declaration or use.

3.5 aon_timer_wdog_lc_escalate_test: plan vs. test

`f_add4ac125a10` · plan vs. test · severity low

Gap: The plan says the test checks that the bark interrupt does not fire during escalation, and that the reset reason proves the wdog was disabled. The test never fails on a bark IRQ (only logs), so the plan's wdog-

disabled claim is not enforced; the escalation reset alone does not show the wdog was suppressed.

Test description says:

- `hw/top_earlgrey/data/ip/chip_aon_timer_testplan.hjson:122-123`: `fail the test in the interrupt handler in case the bark interrupt is fired.` — The test fails if the bark interrupt fires in the interrupt handler.
- `hw/top_earlgrey/data/ip/chip_aon_timer_testplan.hjson:128-129`: `After the reset ensure that the reset cause was due to the escalation to prove that the wdog was disabled.` — After reset, the reset cause is due to the escalation, proving the wdog was disabled.

Test does:

- `sw/device/tests/aon_timer_wdog_lc_escalate_test.c:89`: `LOG_ERROR("Unexpected aon timer interrupt %d", irq);` — check; does not fail the test; An AON timer interrupt is unexpected. It is only logged as an error and not otherwise checked.
- `sw/device/tests/aon_timer_wdog_lc_escalate_test.c:283`: `} else if (rst_info == kDifRstmgrResetInfoEscalation) {` — check; After the second boot the reset reason must be escalation. The test returns true on it and false for any other reason.
- `sw/device/tests/aon_timer_wdog_lc_escalate_test.c:90`: `return true;` — check; After the second boot the reset reason must be escalation. The test returns true on it and false for any other reason.
- `sw/device/tests/aon_timer_wdog_lc_escalate_test.c:288`: `LOG_ERROR("Unexpected rst_info=0x%x", rst_info);` — check; After the second boot the reset reason must be escalation. The test returns true on it and false for any other reason.
- `sw/device/tests/aon_timer_wdog_lc_escalate_test.c:123`: `return false;` — check; After the second boot the reset reason must be escalation. The test returns true on it and false for any other reason.

Verified: The ISR for an AON timer interrupt only calls LOG_ERROR and returns true. It never fails the test, so the plan's "fail the test in the interrupt handler" is not implemented. The reset-reason check does exist, but a wdog bite would also produce a wrong reset reason. A bark alone is not enforced. The finding is low severity: an unhandled, uncleared bark would likely cause a hang or timeout, and a later bite would be caught by the reset-reason CHECK.

Searched:

- Read the full test source and the plan item in `sw/device/tests/aon_timer_wdog_lc_escalate_test.c, chip_aon_timer_testplan.hjson`: The plan item is explicitly linked to this test via bazel and tests fields. The AON ISR only logs.

---

4. Spec and test disagree

No verified findings.

5. Spec not tested

5.1 Partly tested: 3 requirements in aon_timer.hjson > wakeup_list (and 2 more sections)

`f_dcc2e1bc4337` · severity high

1. **Requirement** `cr_5bd774f15d44`: wkup_req is raised when the wakeup timer or the watchdog timer has hit the specified threshold.
   Section: hw/ip/aon_timer/data/aon_timer.hjson > aon_timer > wakeup_list

   - `hw/ip/aon_timer/data/aon_timer.hjson:63`: `Raised if the wakeup or watchdog timer has hit the specified threshold.`

   Tested by `aon_timer_smoke_min_thold`**:** The smoke sequence runs the wakeup and watchdog timers with thresholds constrained to zero. The listed scoreboard checkers check the wakeup request pin after a WKUP threshold hit (c6) and after a watchdog bark (c9).
   Not tested: Only the minimum (zero) threshold value is exercised. Thresholds at other values are not exercised in this test. The pin checks are tied to the scoreboard's predictions rather than a direct threshold comparison.

   Tested by `aon_timer_smoke_max_thold`**:** The smoke sequence runs the wakeup and watchdog timers with thresholds constrained to their maximum values. The listed scoreboard checkers check the wakeup request pin after a WKUP threshold hit (c6) and after a watchdog bark (c9).
   Not tested: Only the maximum threshold value is exercised. The counter may never reach it within the simulation, so the raise of wkup_req at the threshold may not be observed. Mid-range thresholds are not covered.

   Verified: Only min (zero) and max thresholds are constrained; no mid-range threshold test was found.

2. **Requirement** `cr_7f62f6f22c9d`: Once regwen is cleared, the watchdog configuration is locked until the next reset.
   Section: hw/ip/aon_timer/data/aon_timer.hjson > aon_timer > WDOG_REGWEN

   - `hw/ip/aon_timer/data/aon_timer.hjson:259`: `Once cleared, the watchdog configuration will be locked until the next reset.`

   Tested by `WatchdogLockTest.Success`**:** The DIF unit test checks that watchdog_lock writes 0 to WDOG_REGWEN, which is the action that locks the configuration. It does not check that any configuration is actually locked afterwards.
   Not tested: Hardware behavior after regwen is cleared: that writes to the watchdog configuration registers are blocked until reset.

   Tested by `WatchdogStartTest.Locked`**:** With REGWEN mocked to 0, the DIF watchdog_start returns kDifLocked. This verifies software-level lock handling only.
   Not tested: Hardware enforcement of the lock (register writes ignored while regwen is 0) and persistence until reset; only a mocked DIF return value is checked.

   Tested by `WatchdogStopTest.Locked`**:** With REGWEN mocked to 0, the DIF watchdog_stop returns kDifLocked. This is a software-level lock check only.
   Not tested: Hardware enforcement of the lock on the watchdog registers and persistence until reset.

   Tested by `WatchdogRestartTest.Locked`**:** With REGWEN mocked to 0, the DIF watchdog_restart returns kDifLocked. This is a software-level lock check only.
   Not tested: Hardware enforcement of the lock on the watchdog registers and persistence until reset.

   Tested by `WatchdogIsLockedTest.SuccessLocked`**:** With REGWEN mocked to 0, the DIF is_locked reports true. This only checks software interpretation of the regwen value.

Not tested: Hardware enforcement of the lock on the watchdog configuration and persistence until reset.

Verified: The regwen write is commented out in the base vseq, so no DV test locks the watchdog and checks that writes are blocked.

- `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:176-177`: `// csr_utils_pkg::csr_wr(ral.wdog_regwen, wdog_regwen);` — WDOG_REGWEN is never written by the timer sequences; the write is commented out.

3. **Requirement** `cr_4f0006fef572`: The cause bit indicates that the AON timer requested a wakeup. Section: hw/ip/aon_timer/data/aon_timer.hjson > aon_timer > WKUP_CAUSE

   - `hw/ip/aon_timer/data/aon_timer.hjson:363`: `AON timer requested wakeup`

   Tested by `WakeupStatusTest.GetSuccess`**:** Mocks a WKUP_CAUSE read of 1 and checks that the DIF reports cause as true. This verifies only the software reading of the bit.
   Not tested: Hardware behavior: that the cause bit is actually set when the AON timer requests a wakeup. The register is mocked and no timer runs.

   Tested by `aon_timer_smoke`**:** Enables the wakeup timer and waits for expiry. Scoreboard checkers predict wkup_cause and check the wakeup request pin is 1. The wkup_cause readback check applies only if the cause register is read.
   Not tested: The shown stimulus never reads WKUP_CAUSE, so the bit's value is not explicitly checked after the wakeup request. The wakeup-timer path is exercised, not the watchdog-bark source.

   Verified: The sequences only write wkup_cause; they never read it after expiry, and the watchdog-bark source is not explicitly checked.

   - `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:131`: `csr_utils_pkg::csr_wr(ral.wkup_cause, 1'b0);` — wkup_cause is only cleared by the sequences; no read after expiry was found.

Searched:

- wkup_cause and wdog_regwen usage in the DV env in `hw/ip/aon_timer/dv`: The regwen write is commented out; wkup_cause is only written, with no read check after expiry in the sequences.
- csr_rd or regwen in seq_lib in `hw/ip/aon_timer/dv/env/seq_lib`: Only intr_state is read; no WKUP_CAUSE read and no regwen lock test.

Looked at and ruled out:

- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:764-771`: The wkup_cause readback check exists in the scoreboard, but no timer sequence reads the register after the wakeup request.

5.2 Partly tested: 2 requirements in programmers_guide.md > Initialization (and 2 more sections)

`f_2d5f36dd3285` · severity high

1. **Requirement** `cr_d3b9d9ce5458`: Software can lock the watchdog configuration by writing 0 to the regwen bit in WDOG_REGWEN.

Section: hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Initialization

- `hw/ip/aon_timer/doc/programmers_guide.md:9`: `If desired, lock the watchdog configuration by writing 0 to the `regwen` bit in WDOG_REGWEN.`

Tested by `WatchdogLockTest.Success`**:** The DIF lock call is checked to write 0 to WDOG_REGWEN and return OK, which is the software side of locking.
Not tested: Does not verify that the hardware then makes the watchdog configuration registers read-only after the regwen write.

Tested by `WatchdogStartTest.SuccessPauseInSleepAndLock`**:** Watchdog start with the lock option is checked to write 0 to WDOG_REGWEN after enabling.
Not tested: Does not verify that the hardware actually blocks later writes to the watchdog configuration.

Tested by `WatchdogStartTest.SuccessLock`**:** Watchdog start with lock true is checked to write 0 to WDOG_REGWEN.
Not tested: Does not verify that the hardware enforces the lock on the configuration registers.

Verified: The REGWEN write is commented out in the vseq, so lock enforcement is not tested. The scoreboard only predicts the register.

- `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:176-177`: `// csr_utils_pkg::csr_wr(ral.wdog_regwen, wdog_regwen);` — WDOG_REGWEN write is commented out, so the lock is never exercised

2. **Requirement** `cr_ebc0e38e9d47`: For target values close to overflow, software disables the timer via WKUP_CTRL while writing the counter.
Section: hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Wakeup count and threshold access > Writing the counter

   - `hw/ip/aon_timer/doc/programmers_guide.md:56`: `For such values, disable the timer with WKUP_CTRL while writing the counter.`

   Tested by `WakeupRestartTest.Success`**:** The DIF wakeup_restart test expects WKUP_CTRL enable to be cleared, then the WKUP count registers to be written, then enable to be set again. That is the disable-write-enable pattern the requirement describes.
   Not tested: It only writes a count of 0, not a value close to overflow. It does not check the selective use of the disable for near-overflow values, and it does not check that the counter value is uncorrupted.

   Tested by `WakeupStartTest.Success`**:** The DIF wakeup_start test expects WKUP_CTRL enable to be cleared before the count and threshold registers are written, and the timer to be enabled afterwards. This is a disable-while-writing-the-counter sequence.
   Not tested: It only writes a count of 0, not a near-overflow value. It does not check that software disables the timer specifically for such values.

   Verified: No DV test disables the timer, writes a near-overflow count, and re-enables. Counts are constrained near the threshold, not near rollover.

Items 3, 4, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- wkup_cause, regwen, wkup_count, wkup_ctrl in aon_timer DV in `hw/ip/aon_timer/dv`: Scoreboard checks wkup_cause clear and the wkup_req pin. shutdown clears WKUP_CTRL enable and WKUP_CAUSE. Count writes are done with the timer enabled. The regwen write is commented out. No near-overflow disable-and-write test.

Looked at and ruled out:

- `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:131`: Writes 0 to WKUP_CAUSE in shutdown; the scoreboard check on it is what refutes item cr_0838c215b7f8.

5.3 Partly tested: 2 requirements in theory_of_operation.md > Theory of Operation (and 1 more section)

`f_517767a1d7fa` · severity high

1. **Requirement** `cr_20f39fd89cf5`: Both timers halt counting when lc_escalate_en_i is set.
   Section: hw/ip/aon_timer/doc/theory_of_operation.md > Theory of Operation

   - `hw/ip/aon_timer/doc/theory_of_operation.md:5`: `Both timers halt the counting if the signal `lc_escalate_en_i` is set`

   Tested by `aon_timer_wdog_lc_escalate_test`**:** The test arms the AON watchdog to bark and bite during escalation phase 1, then forces escalation. After the escalation reset it checks that the reset reason is escalation. That outcome would differ if the watchdog kept counting through lc escalation and caused its own reset.
   Not tested: The wakeup timer halting is never exercised or checked. The watchdog halt is only inferred from the reset reason, because an unexpected AON timer interrupt is only logged (k6 only reports).

   Verified: No DV or software test checks that the counters stay halted while lc_escalate_en_i is set.

   - `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:208`: `cfg.lc_escalate_en_vif.drive(0);` — DV only drives escalation at reset and then disables it, so no test checks that counting halts during escalation.

2. **Requirement** `cr_f75f4b79d7eb`: A register write can complete on the bus before the data has reached the underlying register in the AON domain.
   Section: hw/ip/aon_timer/doc/theory_of_operation.md > Theory of Operation > Design Details

   - `hw/ip/aon_timer/doc/theory_of_operation.md:26`: `This means that writes can complete before the data has reached its underlying register in the slow clock domain.`

   Tested by `aon_timer_prescaler`**:** The test has to spin-wait on a backdoor read until the written prescaler appears in the AON register, which relies on the write reaching the AON domain after the bus write was issued.
   Not tested: It does not check that the bus write completes before the data reaches the AON register. There is no timing comparison between bus completion and AON-register update.

   Verified: No check compares bus write completion with the AON register update.

Items 2, 3, 4 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- escalation handling in scoreboard and sequences in `hw/ip/aon_timer/dv`: The scoreboard has no escalate check. Sequences only drive lc_escalate_en at reset and disable it afterwards.
- threshold and prescaler checking in `scoreboard and vseqs`: The scoreboard models the 64-bit threshold and the prescaler scaling of expiry time.
- bus write completion compared with AON register update in `hw/ip/aon_timer/dv`: Nothing found. The scoreboard only waits for the write-enable pulse in the AON domain.

Looked at and ruled out:

- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:390-412`: It waits for the AON write-enable pulse before counting, but it never checks that the bus write completed earlier.

5.4 No test found: 1 requirement in aon_timer.hjson > inter_signal_list (and 2 more sections)

`f_c027da08f1ba` · severity medium

1. **Requirement** `cr_d3bb7e9459e0`: The policy selection vector parameter selects the RACL policy for each register.
   Section: hw/ip/aon_timer/data/aon_timer.hjson > aon_timer > inter_signal_list

   - `hw/ip/aon_timer/data/aon_timer.hjson:116`: `The policy selection vector (parameter) selects the policy for each register.`
   - `hw/ip/aon_timer/doc/interfaces.md:20`: `The policy selection vector (parameter) selects the policy for each register.`

   Verified: No RACL test; policies tied to 0.

   - `hw/ip/aon_timer/dv/tb.sv:25-26` — RACL policies tied to 0; no RACL stimulus

     ```
     racl_policy_vec_t racl_policies;
       assign racl_policies = 0;
     ```

Items 2, 3, 4, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- racl|intr_test in `hw/ip/aon_timer/dv`: racl only tied off in tb; intr_test modelled in scoreboard and imported test
- prescaler qe in `hw/ip/aon_timer`: core resets prescale counter on qe; no explicit check traced

---

5.5 No test found: 1 requirement in interfaces.md > Inter-Module Signals

`f_4f056852409f` · severity medium

1. **Requirement** `cr_cd5358219466`: racl_policies is a top_racl_pkg::racl_policy_vec unidirectional input received by the module, carrying the RACL policy vector from a racl_ctrl instance.

   - `hw/ip/aon_timer/doc/interfaces.md:15`: `1`
   - `hw/ip/aon_timer/doc/interfaces.md:20`: `racl_policies`
   - `hw/ip/aon_timer/doc/interfaces.md:20`: `top_racl_pkg::racl_policy_vec`
   - `hw/ip/aon_timer/doc/interfaces.md:20`: `uni`

- `hw/ip/aon_timer/doc/interfaces.md:20`: `rcv`
- `hw/ip/aon_timer/doc/interfaces.md:20`: `Incoming RACL policy vector from a racl_ctrl instance.`

Verified: No test drives or checks racl_policies; the testbench ties it to 0.

- `hw/ip/aon_timer/dv/tb.sv:25-26` — Policy vector is constant zero; no stimulus

```
racl_policy_vec_t racl_policies;
assign racl_policies = 0;
```

Searched:

- racl in aon_timer/dv in `hw/ip/aon_timer/dv`: only tb.sv tie-off and an exclusion file
- racl in top_earlgrey/dv in `hw/top_earlgrey/dv`: nothing

Looked at and ruled out:

- `hw/ip/aon_timer/dv/aon_timer_sim_cfg.hjson:47`: Only a coverage exclusion file; not a test.

5.6 No test found: 1 requirement in interfaces.md > Interfaces (and 1 more section)

`f_edad66225369` · severity medium

4. **Requirement** `cr_a34add480316`: racl_error is a top_racl_pkg::racl_error_log unidirectional output driven by the module, carrying the RACL error log information of this module.
   Section: hw/ip/aon_timer/doc/interfaces.md > Interfaces > Inter-Module Signals

   - `hw/ip/aon_timer/doc/interfaces.md:15`: `1`
   - `hw/ip/aon_timer/doc/interfaces.md:21`: `racl_error`
   - `hw/ip/aon_timer/doc/interfaces.md:21`: `top_racl_pkg::racl_error_log`
   - `hw/ip/aon_timer/doc/interfaces.md:21`: `uni`
   - `hw/ip/aon_timer/doc/interfaces.md:21`: `req`
   - `hw/ip/aon_timer/doc/interfaces.md:21`: `RACL error log information of this module.`

   Verified: racl_error_o is unconnected in the TB and no check covers it.

   - `hw/ip/aon_timer/dv/tb.sv:62`: `.racl_error_o (),` — racl_error output unconnected in the TB, so it is never checked.

Items 1, 2, 3, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- sleep_mode and racl_error in the aon_timer dv directory in `hw/ip/aon_timer/dv`: sleep_mode is driven and modelled; racl_error_o is unconnected and appears only in coverage exclusions

Looked at and ruled out:

- `hw/ip/aon_timer/dv/cov/aon_timer_unr_non_racl_manually_excluded_excl.el:23-45`: These are coverage exclusions for racl_error toggles, not checks.

5.7 No test found: 2 requirements in programmers_guide.md > Writing the threshold (and 1 more section)

`f_b7844c8b9c9f` · severity medium

1. **Requirement** `cr_ab0d74b3889d`: Enable the wakeup interrupt after the threshold update is complete.
   Section: hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Wakeup count and threshold access > Writing the threshold

   - `hw/ip/aon_timer/doc/programmers_guide.md:81`: `enable_wakeup_interrupt();`

   Verified: No test enables the wakeup interrupt after a threshold update; intr_enable does not appear in the DV sequences.

   - `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:181-183`: `csr_utils_pkg::csr_wr(ral.intr_state, 2'b01);` — wkup_init clears intr_state and then writes the thresholds. It never disables or enables intr_enable around the threshold writes.

2. **Requirement** `cr_cd988f27543a`: Disable the wakeup interrupt before writing the threshold registers.
   Section: hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Wakeup count and threshold access > Writing the threshold

   - `hw/ip/aon_timer/doc/programmers_guide.md:68`: `disable_wakeup_interrupt();`

   Verified: No test disables the wakeup interrupt before writing the threshold registers; wkup_init writes them directly.

Item 3 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- intr_enable used in aon_timer DV sequences in `hw/ip/aon_timer/dv/env/seq_lib`: no matches
- retrigger / remains above threshold in scoreboard in `aon_timer_scoreboard.sv`: no explicit retrigger check

---

5.8 No test found: 5 requirements in programmers_guide.md > Wakeup count and threshold access (and 2 more sections)

`f_37f49dda2889` · severity medium

1. **Requirement** `cr_f0d34bc61b8f`: The 64-bit wakeup counter and threshold values cannot be read or modified in a single atomic access.
   Section: hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Wakeup count and threshold access

   - `hw/ip/aon_timer/doc/programmers_guide.md:18`: `It is not possible to read or modify the 64-bit values in a single atomic access.`
   - `hw/ip/aon_timer/doc/theory_of_operation.md:33`: `It is not possible to do a single atomic read or write of the 64-bit values.`

   Verified: No test or check covers non-atomic 64-bit access.

- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:784-786`: `wkup_thold[31:0] = csr.get_mirrored_value();` — The scoreboard only mirrors the threshold to predict interrupt timing. It has no check that hardware leaves the threshold unmodified and no check on atomicity.

2. **Requirement** `cr_13a3d2eb44cf`: Hardware does not modify WKUP_THOLD_LO and WKUP_THOLD_HI.
   Section: hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Wakeup count and threshold access > Reading the threshold

   - `hw/ip/aon_timer/doc/programmers_guide.md:60`: `The hardware does not alter [`WKUP_THOLD_LO`](registers.md#wkup_thold_lo) and [`WKUP_THOLD_HI`](registers.md#wkup_thold_hi)`

   Verified: No explicit check that hardware leaves the threshold unchanged. The scoreboard only uses the mirrored value.

3. **Requirement** `cr_40aa3bfcebb9`: Write WKUP_THOLD_HI with the upper 32 bits of the new threshold second.
   Section: hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Wakeup count and threshold access > Writing the threshold

   - `hw/ip/aon_timer/doc/programmers_guide.md:74`: `// Set the upper word to its desired value. The resulting threshold will still be at least as big`
   - `hw/ip/aon_timer/doc/programmers_guide.md:76`: `REG_WRITE(WKUP_THOLD_HI, new_thold >> 32);`

   Verified: DV does not follow the guide's write sequence, and nothing checks the intermediate state.

   - `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:193-196`: `write_wkup_reg(ral.wkup_thold_lo, wkup_thold[31:0]);` — DV writes LO then HI with the final values directly. It does not first write LO=UINT32_MAX, then HI, then LO, as the guide describes.

4. **Requirement** `cr_64e15132845f`: Write WKUP_THOLD_LO with the lower 32 bits of the new threshold third.
   Section: hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Wakeup count and threshold access > Writing the threshold

   - `hw/ip/aon_timer/doc/programmers_guide.md:78-79`: `// Set the lower word to its desired value. REG_WRITE(WKUP_THOLD_LO, new_thold & 0xffff_ffff);`

   Verified: The LO write happens, but not as the final step of the guide's sequence, and nothing checks the sequence's property.

5. **Requirement** `cr_7e6da13c8b08`: Write WKUP_THOLD_LO with its maximum value (UINT32_MAX) first.
   Section: hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Wakeup count and threshold access > Writing the threshold

   - `hw/ip/aon_timer/doc/programmers_guide.md:70`: `// Set the lower word to its maximum value. The resulting threshold will definitely be at least as`
   - `hw/ip/aon_timer/doc/programmers_guide.md:72`: `REG_WRITE(WKUP_THOLD_LO, UINT32_MAX);`

   Verified: No test writes UINT32_MAX to LO first as a safe-update step.

Searched:

- threshold register usage in the aon_timer dv and data directories (atomic access, item 1) in `hw/ip/aon_timer/{dv,data}`: Only the randomized LO/HI writes in the base vseq and scoreboard prediction. No atomicity check.
- hardware-does-not-modify check on the thresholds (item 2) in `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv`: The scoreboard mirrors the thresholds for timing prediction only. No explicit unchanged-threshold check.
- write ordering of THOLD_HI (item 3) in `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv`: HI is written after LO with the final value. It is not the guide's second step.
- write ordering of THOLD_LO as the last step (item 4) in `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv`: LO is written first with the final value. It is not the guide's third step.
- UINT32_MAX written to THOLD_LO first (item 5), plus device software tests in `hw/ip/aon_timer/dv and sw/device/tests`: No such sequence. The only software match for WKUP_THOLD was the unrelated rv_core_ibex_mem_test.c.

Looked at and ruled out:

- `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_smoke_max_thold_vseq.sv:1-23`: Exercises a maximum threshold value, but not the write-ordering sequence.

5.9 No test found: 4 requirements in theory_of_operation.md > Theory of Operation

`f_00c764972edf` · severity medium

1. **Requirement** `cr_517c566609ad`: RACL ports are enabled by setting parameter EnableRacl to 1.

   - `hw/ip/aon_timer/doc/theory_of_operation.md:12`: `In order to enable the RACL ports, the parameter `EnableRacl` should be set to 1.`

   Verified: EnableRacl is not exercised; only a tie-off exists in the testbench.

   - `hw/ip/aon_timer/dv/tb.sv:25-26` — RACL policies are tied to 0; racl_error_o is unconnected, so there is no RACL testing.

     ```
     racl_policy_vec_t racl_policies;
       assign racl_policies = 0;
     ```

2. **Requirement** `cr_6c14ebc81991`: intr_wkup_timer_expired_o, intr_wdog_timer_bark_o and nmi_wdog_timer_bark_o are driven on the SYS domain.

   - `hw/ip/aon_timer/doc/theory_of_operation.md:8`: `The output signals `intr_wkup_timer_expired_o`, `intr_wdog_timer_bark_o` and `nmi_wdog_timer_bark_o` are driven on the SYS domain.`

   Verified: No check of the SYS-domain driving for the interrupt outputs was found.

3. **Requirement** `cr_7aa0f0735984`: wkup_req_o and aon_timer_rst_req_o are driven on the AON domain.

   - `hw/ip/aon_timer/doc/theory_of_operation.md:9`: `The output signals `wkup_req_o` and `aon_timer_rst_req_o` are driven on the AON domain.`

Verified: Values are checked but the AON domain is not.

4. **Requirement** `cr_c98fa20138a9`: The block has RACL ports racl_policies_i and racl_error_o.
   - `hw/ip/aon_timer/doc/theory_of_operation.md:11`: `Furthermore, the block also has RACL ports `racl_policies_i` and `racl_error_o``

   Verified: RACL ports are tied off and unchecked.

Item 2 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- racl and nmi references in aon_timer dv in `hw/ip/aon_timer/dv`: Only tb.sv tie-off, the NMI assertion, and a coverage exclusion file.
- wkup_req / rst_req checks in `hw/ip/aon_timer/dv`: Scoreboard checks values but not the clock domain.

Looked at and ruled out:

- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:1278-1288`: Checks wkup_req value, not the AON clock domain it is driven on.

5.10 Partly tested: 1 requirement in theory_of_operation.md > Design Details

`f_596c7148f2e0` · severity medium

1. **Requirement** `cr_fe9918a1d87a`: Wakeup count bits [31:0] are in WKUP_COUNT_LO.
   - `hw/ip/aon_timer/doc/theory_of_operation.md:32`: `Bits[31:0] in registers`
   - `hw/ip/aon_timer/doc/theory_of_operation.md:32`: `WKUP_COUNT_LO`

   Tested by `WakeupGetCountTest.OverflowSuccess`**:** The mocked DIF read returns the LO register value 0x5A5A5A5A in the low 32 bits of the composed count (0xA5A5A5A65A5A5A5A). That is consistent with WKUP_COUNT_LO holding bits [31:0].
   Not tested: This is a software unit test against mocked register reads. It does not check the hardware register map or that the real WKUP_COUNT_LO register holds count bits [31:0].

   Verified: No hardware-level check found that WKUP_COUNT_LO equals count bits [31:0].

   - `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:997-999`: `csr_rd(.ptr(ral.wkup_count_lo), .value(rtl_count[31:0]), .backdoor(1));` — Backdoor read used only to sample coverage; no comparison or check. The upper half is also read from wkup_count_lo (likely a typo).

Searched:

- WKUP_COUNT_LO usage in DV in `hw/ip/aon_timer/dv`: Scoreboard timing and coverage reads, sequence writes; no check of LO against count bits [31:0]
- WKUP_COUNT_LO usage in sw in `sw/`: Only the DIF, the DIF unit test (mocked) and the host HAL

Looked at and ruled out:

- `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:913-914`: Backdoor reads feed the wake-up timing calculation; they do not check that LO holds count bits [31:0].

5.11 No test found: 2 requirements in theory_of_operation.md > Design Details

`f_f56e7fab84c9` · severity medium

1. **Requirement** `cr_bfbda21a02ad`: When integrating the AON timer, clk_aon_i must be started before clk_i.

   - `hw/ip/aon_timer/doc/theory_of_operation.md:36`: `When integrating the AON timer into a bigger design, you must start the always-on clock (clk_aon_i) before you start the main clock (clk_i).`

   Verified: No test or assertion covers the clk_aon_i-before-clk_i start order.

2. **Requirement** `cr_c24c1a80fbfe`: Reading back a register guarantees that the preceding write to it has completed.

   - `hw/ip/aon_timer/doc/theory_of_operation.md:27`: `If software needs to guarantee completion of a register write then it can read back the register value.`

   Verified: No test reads back a register to confirm earlier writes completed. The scoreboard only checks the timing of the write updates.

   - `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:462`: `task aon_timer_scoreboard::update_wkup_thold_lo_timely();` — The scoreboard checks the timing of thold_lo register updates. It does not check that a read-back guarantees completion of earlier writes.

Item 2 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- wkup_thold_lo, clock start order, busy and read-back in aon_timer dv in `hw/ip/aon_timer/dv`: Only scoreboard thold_lo timing updates and CDC busy coverage exclusions. No clock start-order test and no read-back-guarantees-completion check.

5.12 No test found: 2 requirements in theory_of_operation.md > Design Details

`f_fce24c4eba49` · severity medium

1. **Requirement** `cr_20034a8664a3`: The always-on timer runs on a clock of approximately 200KHz.

   - `hw/ip/aon_timer/doc/theory_of_operation.md:20`: `The always-on timer will run on a ~200KHz clock.`

   Verified: The clock is only configured to 200kHz in tb.sv and nothing checks it.

   - `hw/ip/aon_timer/dv/tb.sv:78`: `aon_clk_rst_if.set_freq_khz(200);` — The 200kHz clock is only configured here, and nothing checks it.

2. **Requirement** `cr_8bcb4d2923d5`: A register read-back guarantees completion of all previous writes to the peripheral.

   - `hw/ip/aon_timer/doc/theory_of_operation.md:28`: `This will guarantee the completion of all previous writes to the peripheral.`

   Verified: No test reads a register back to check completion of earlier writes. The helper waits for the write-enable pulse instead.

   - `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:275`: `csr_utils_pkg::csr_wr(ptr, value);` — The write helper waits for the write-enable pulse and does not do a read-back completion check.

Items 2, 3, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- Tests, sequences and scoreboard handling of WKUP_THOLD_HI, widths, clock frequency and read-back in `hw/ip/aon_timer/dv`: The scoreboard handles wkup_thold_hi and the sequences write bits [63:32]. The clock is set to 200kHz in tb.sv. No read-back completion check was found.

Inconclusive

The verifier could neither confirm nor refute these. They need a human decision.

- `f_517767a1d7fa` (partly tested) TL-UL register accesses in the SYS domain are synchronized to the AON slow clock using the async register generation feature. — Many registers are written across the CDC and checked by the scoreboard, but I found no explicit test of the async synchronization read path.
- `f_c027da08f1ba` (no test found) WKUP_CTRL.prescaler: hardware receives a write-qualifier pulse on software writes (true). — Indirectly covered by the prescaler vseq; no explicit qe check traced.
- `f_edad66225369` (no test found) The module has no TL-UL bus host interfaces. — This is a structural absence of a bus host interface, with nothing specific to test.
- `f_edad66225369` (no test found) The module has no peripheral pins for chip IO. — This is a structural absence of chip IO pins, with nothing specific to test.
- `f_edad66225369` (no test found) The module has primary clock input clk_i. — clk_i is used by every test, but no dedicated check exists.
- `f_b7844c8b9c9f` (no test found) The interrupt retriggers at the next clock tick if the count remains above the threshold after clearing and the timer remains enabled. — The scoreboard models count above threshold as immediate expiry, but I did not find a test that clears the interrupt while the count stays above the threshold and then checks for a retrigger at the next tick.
- `f_f56e7fab84c9` (no test found) Wakeup threshold bits [31:0] are in the low threshold register. — The scoreboard exercises the thold_lo register implicitly, but no check explicitly targets the bits[31:0] mapping. The spec line also has a typo.
- `f_3764d4badb94` (test plan items with no test) dv_base_lockable_field_cov — Coverage item with regwen-locked fields; collected in all tests, but no dedicated test.
- `f_3764d4badb94` (test plan items with no test) mem_walk — Generic mem_walk test is imported, but the design has no memories, so it does nothing.
- `f_6037878d66f2` (test plan items with no test) max_outstanding_cg — max_outstanding_cg is sampled by the shared TL monitor in every test, but it is coverage rather than a check. The env sets

max_outstanding_req to 1.

- `f_6037878d66f2` (test plan items with no test) pending_req_on_rst_cg — pending_req_on_rst_cg is sampled by the shared TL monitor on reset. No aon_timer test specifically drives a pending TL access across reset.
- `f_a3cc6b93921a` (in a test plan, but no test found) Each write to WKUP_CTRL resets the internal prescaler count. — No explicit check exists. The scoreboard's count reset and timing prediction might catch a wrong reset, but this is not confirmed.
- `f_d2255379ea1e` (test plan items with no test) mem_partial_access — The generic mem_partial_access test is imported by aon_timer, but the block has no memory, so it is unclear what the plan item means for this block.

Withdrawn

Candidates the verifier refuted, with the reason. Listed so the refutations can be checked too.

- `f_4f700106f7fc` (spec too ambiguous to test) A watchdog bark interrupt is generated when the watchdog count reaches the threshold value. — The aon_timer README states the bark condition: a bark fires on every AON clock tick when the count is at least the threshold. (`hw/ip/aon_timer/README.md:55`)
- `f_087fbd904483` (spec too ambiguous to test) The count at which a wakeup interrupt should be generated, top 32 bits. — Other spec text answers the comparison and immediate-firing questions: the README says the outputs go high when the count is at or above the threshold, on a prescaler tick. (`hw/ip/aon_timer/README.md:35`, `hw/ip/aon_timer/doc/programmers_guide.md:93`)
- `f_1bc5edd4d321` (spec too ambiguous to test) The count at which a wakeup interrupt should be generated, bottom 32 bits. — The aon_timer README says the comparison is "at or above", so it is >=. (`hw/ip/aon_timer/README.md:35`, `hw/ip/aon_timer/doc/programmers_guide.md:93`)
- `f_2d4541f82cf3` (spec too ambiguous to test) wkup_req is raised when the wakeup timer or the watchdog timer has hit the specified threshold. — The aon_timer README states that wkup_req_o is raised by the wakeup timer or a watchdog bark, and that it stays high until software writes 0 to WKUP_CAUSE. (`hw/ip/aon_timer/README.md:75-76`, `hw/ip/aon_timer/README.md:41-42`)
- `f_8da2bbc84b1a` (spec too ambiguous to test) The wakeup timer pre-scaler can slow the count so the counter matches a desired time unit. — Other spec text states both the field width (12 bits) and the division rule (cycles per tick = prescaler + 1). (`hw/ip/aon_timer/README.md:32`, `hw/ip/aon_timer/doc/registers.md:57`)
- `f_6bf289f534a0` (spec contradicts itself) TL-UL register accesses in the SYS domain are synchronized to the AON slow clock using the async register generation feature. — Line 25 says reads are synchronized. (`hw/ip/aon_timer/doc/theory_of_operation.md:25-28`)
- `f_8b2618a31c84` (spec not tested) 5 plan item(s) with no implementing test — All five items are implemented. (`hw/dv/tools/dvsim/tests/tl_access_tests.hjson:20-23`, `hw/dv/tools/dvsim/tests/csr_tests.hjson:37-39`)
- `f_e193da4cd3c1` (spec not tested) 5 plan item(s) with no implementing test — All five items are implemented through shared CIP infrastructure that aon_timer imports. (`hw/dv/tools/dvsim/tests/alert_test.hjson:14-19`, `hw/dv/sv/cip_lib/seq_lib/cip_base_vseq.sv:960`)
- `f_b715d5bebccc` (spec not tested) Untested: 1 requirement(s) in hw/ip/aon_timer/data/aon_timer.hjson > aon_timer > INTR_TEST — Tests do write INTR_TEST.wdog_timer_bark, and the scoreboard checks

the resulting effect. (`hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:841-847`, `hw/ip/aon_timer/dv/env/seq_lib/aon_timer_custom_intr_vseq.sv:38`)

- `f_7df3b4d77206` (spec not tested) Planned but not implemented: 5 requirement(s) in hw/ip/aon_timer/doc/interfaces.md > Interfaces … (4 sections) — The aon_timer sim cfg imports the shared tl_access_tests.hjson (providing {name}_tl_intg_err with +run_tl_intg_err and {name}_tl_errors), csr_tests, sec_cm_tests, and the earlgrey chip cfg has chip_sw_aon_timer_wdog_lc_escalate. (`hw/ip/aon_timer/dv/aon_timer_sim_cfg.hjson:34-35`, `hw/dv/tools/dvsim/tests/tl_access_tests.hjson:20-23`)
- `f_b1041443036d` (spec not tested) Partially tested: 1 requirement(s) in hw/ip/aon_timer/doc/programmers_guide.md > Programmer's Guide > Interrupt Handling — The finding's gap is that re-enabling WKUP_CTRL after a disable is never exercised. (`hw/ip/aon_timer/dv/env/seq_lib/aon_timer_alternating_enable_on_off_vseq.sv:37-42`, `hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:910-913`)
- `f_2fc38343b1ac` (spec not tested) Register reset values: no suite covers every register — A generic csr_hw_reset test exists in the shared dvsim library, and the AES sim config (AES is my best guess at the block) imports it. (`hw/dv/tools/dvsim/tests/csr_tests.hjson:21-23`, `hw/dv/tools/dvsim/testplans/csr_testplan.hjson:9`)
- `f_e684b2415bb9` (spec not tested) Register access: no suite covers every register — The shared dvsim csr_tests.hjson defines generic csr_rw and csr_bit_bash tests, and many IP sim configs import it, including aes, hmac and kmac. (`hw/dv/tools/dvsim/tests/csr_tests.hjson:29-33`)
- `f_fe240411fef7` (spec not tested) Register write enable locks: no suite covers every register — The repository's generic CSR tests cover this. (`hw/dv/tools/dvsim/testplans/csr_testplan.hjson:98-99`, `hw/dv/sv/dv_base_reg/dv_base_reg.sv:356-359`)

Withdrawn items from the groups in section 5 (39):

- Writing 0 to WKUP_CAUSE clears the wakeup request when a wakeup event has been caused by the timer. — The scoreboard predicts the WKUP_CAUSE clear and compares wkup_req against it, with expiry stimulated by the smoke tests. (`hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:764-771`)
- Clearing WKUP_COUNT_HI and WKUP_COUNT_LO resets the wakeup timer count. — The DV writes WKUP_COUNT_LO and HI (including zero) and the scoreboard predicts the count and the expiry. (`hw/ip/aon_timer/dv/env/seq_lib/aon_timer_alternating_enable_on_off_vseq.sv:36-41`)
- Clearing the enable bit in WKUP_CTRL disables the wakeup timer. — The alternating enable/disable vseq clears WKUP_CTRL enable, and the scoreboard checks interrupts and wkup_req afterwards. (`hw/ip/aon_timer/dv/env/seq_lib/aon_timer_alternating_enable_on_off_vseq.sv:36-41`)
- The wakeup count and threshold are each 64-bit values accessed through two 32-bit registers (hi and lo). — The base vseq writes a random 64-bit threshold through hi and lo, and the scoreboard checks the expiry against the full 64-bit value. (`hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:933-934`)
- The wakeup timer pre-scaler can slow the count so the counter matches a desired time unit. — The prescaler vseq randomizes the prescaler, and the scoreboard checks the expiry time scaled by (prescaler+1). (`hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:933-934`)
- INTR_TEST.wkup_timer_expired: the register is implemented outside the register block (true). — Scoreboard intr_test model and the generic intr_test test check this. (`hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:829-834`)
- INTR_TEST.wkup_timer_expired: hardware receives a write-qualifier pulse on software writes (true). — The intr_test write must reach intr_state through the qe pulse; the scoreboard checks it. (`hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:829-834`)

- INTR_TEST.wdog_timer_bark: the register is implemented outside the register block (true). — Same as the other intr_test items. (`hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:829-834`)
- sleep_mode is a 1-bit logic unidirectional input received by the module (rcv). — sleep_mode is driven by the sleep interface and the scoreboard models the pause-in-sleep behaviour. (`hw/ip/aon_timer/dv/tb.sv:69`)
- nmi_wdog_timer_bark_o is a copy of intr_wdog_timer_bark_o. — A tb.sv assertion checks nmi equals bark on every run. (`hw/ip/aon_timer/dv/tb.sv:72`)
- Wakeup threshold bits [63:32] are in WKUP_THOLD_HI. — Sequences write wkup_thold_hi and the scoreboard predicts the wakeup behaviour that results. (`hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:80-82`)
- The watchdog timer is 32 bits wide. — Watchdog thresholds are randomised up to the full 32-bit range with the scoreboard checking the results. (`hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:80-82`)
- The wakeup timer is 64 bits wide. — Wakeup thresholds are randomised up to the full 64-bit range with the scoreboard checking the results. (`hw/ip/aon_timer/dv/env/seq_lib/aon_timer_base_vseq.sv:80-82`)
- csr_mem_rw_with_rand_reset — Test csr_mem_rw_with_rand_reset is defined and imported. (`hw/dv/tools/dvsim/tests/csr_tests.hjson:61-63`)
- csr_aliasing — Test csr_aliasing is defined and imported. (`hw/dv/tools/dvsim/tests/csr_tests.hjson:45-47`)
- X-value output assertions — ASSERT_KNOWN assertions in RTL check outputs for X in every sim. (`hw/ip/aon_timer/rtl/aon_timer.sv:291`)
- TLUL protocol assertions — tlul_assert is bound into aon_timer through the sim_tops entry aon_timer_bind, so the assertions run with every test and fail it on a protocol violation. (`hw/ip/aon_timer/dv/sva/aon_timer_bind.sv:7-9`)
- The module has a TL-UL bus device interface named tl. — csr_rw and tl_access tests are imported in the aon_timer sim cfg. (`hw/ip/aon_timer/dv/aon_timer_sim_cfg.hjson:34-35`)
- tl is a tlul_pkg::tl request/response interface on which the module acts as responder (rsp). — tl_errors/tl_access tests are imported, and the tb binds the TL assertions. (`hw/ip/aon_timer/dv/aon_timer_sim_cfg.hjson:34-35`)
- lc_escalate_en is a 1-bit lc_ctrl_pkg::lc_tx unidirectional input received by the module (rcv). — The chip-level SW test chip_sw_aon_timer_wdog_lc_escalate exists, and the DV tb drives lc_escalate_en. (`hw/top_earlgrey/dv/chip_sim_cfg.hjson:1250-1252`)
- The fatal_fault alert is triggered when a fatal TL-UL bus integrity fault is detected. — aon_timer_tl_intg_err test is imported. (`hw/ip/aon_timer/dv/aon_timer_sim_cfg.hjson:34-35`)
- The AON_TIMER.BUS.INTEGRITY countermeasure provides an end-to-end bus integrity scheme. — The tl_intg_err and sec_cm tests are imported. (`hw/ip/aon_timer/dv/aon_timer_sim_cfg.hjson:34-35`)
- tl_intg_err — aon_timer_tl_intg_err comes from the imported tl_access_tests.hjson. (`hw/dv/tools/dvsim/tests/tl_access_tests.hjson:20-23`)
- intr_test_cg — Covergroup sampled in the scoreboard; it is not a test to implement. (`hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:355`)
- csr_bit_bash — aon_timer_csr_bit_bash comes from the imported csr_tests.hjson. (`hw/dv/tools/dvsim/tests/csr_tests.hjson:37-39`)
- sec_cm_bus_integrity — The sec_cm testplan maps to aon_timer_tl_intg_err, which exists via tl_access_tests.hjson. (`hw/dv/tools/dvsim/tests/tl_access_tests.hjson:20-23`)

- intr_pins_cg — Covergroup sampled in the scoreboard at line 355. (`hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:355`)
- AON_TIMER.WATCHDOG.DISABLE_BY_LC_CTRL: the watchdog is disabled when the life cycle controller escalates. — The chip-level test chip_sw_aon_timer_wdog_lc_escalate is in the regression and implemented. (`hw/top_earlgrey/dv/chip_sim_cfg.hjson:1250-1252`)
- Setting the enable bit in WKUP_CTRL re-enables the wakeup timer. — The alternating_enable_on_off test covers re-enable after a disable. (`hw/ip/aon_timer/dv/env/seq_lib/aon_timer_alternating_enable_on_off_vseq.sv:37-42`)
- INTR_TEST.wdog_timer_bark: hardware receives a write-qualifier pulse on software writes (true). — Stimulus writes intr_test; the scoreboard predicts intr_state and the interrupt output and checks them. (`hw/ip/aon_timer/dv/env/aon_timer_scoreboard.sv:841-847`)
- csr_hw_reset — aon_timer_csr_hw_reset is defined in the imported csr_tests.hjson and is run with the +csr_hw_reset option. (`hw/dv/tools/dvsim/tests/csr_tests.hjson:21-23`)
- tl_intg_err_cg_wrap — This is a covergroup, not a test. (`hw/dv/sv/cip_lib/cip_base_env_cov.sv:135`)
- regwen_val_when_new_value_written_cg — This is a covergroup, not a test. (`hw/dv/sv/dv_base_reg/dv_base_lockable_field_cov.sv:19`)
- bit_toggle_cg_wrap — This is a covergroup, not a test. (`hw/dv/sv/cip_lib/cip_base_env_cov.sv:135`)
- tl_a_chan_cov_cg — Coverage is collected by the tl_agent_cov coverage group during the tl_access tests, so it is not a test that must exist. (`hw/dv/sv/cip_lib/cip_base_scoreboard.sv:59-60`)
- tl_errors_cg_wrap — tl_errors_cg_wrap is instantiated in the shared scoreboard. (`hw/dv/sv/cip_lib/cip_base_scoreboard.sv:59-60`)
- alert_test — aon_timer_alert_test runs the common vseq, which has checks. (`hw/dv/tools/dvsim/tests/alert_test.hjson:14-19`)
- tl_intg_err_cg — tl_intg_err_cg is sampled by the shared scoreboard. (`hw/dv/sv/cip_lib/cip_base_scoreboard.sv:59-60`)
- regwen_csr_and_corresponding_lockable_csr — WDOG_REGWEN is covered by the generic csr_rw tests; I did not trace the lockable-CSR check in detail. (`hw/dv/tools/dvsim/tests/alert_test.hjson:14-19`)

Run record

Stage	Model
Spec extraction	claude-sonnet-5-5
Test extraction	claude-sonnet-5-5
Merge and link	claude-sonnet-5-5
Correlate	claude-sonnet-5-5
Analyze	claude-opus-5-5, claude-sonnet-5-5
Verify	claude-sonnet-5-5

This run: $7.16 in 1079 s (cached results reused where the inputs were unchanged). All LLM spend recorded for this workspace: $7.16.

Manifest `work/opentitan-aon-timer/manifest.yaml` (sha256 `d1ae677b9090`); findings `work/opentitan-aon-timer/findings.yaml`; verifier transcripts in `work/opentitan-aon-timer/verify_runs/<finding id>/`.